

Lecture 10: Game Theory & Adversarial Search

IT5052 - Fundamentals of Artificial Intelligence

Jeewaka Perera

Department of Computer Science, SLIIT
MSc. in AI

Session Objectives

By the end of this lecture, you will be able to:

- Understand the foundations of **Game Theory** and Nash Equilibria.
- Differentiate between **Non-Zero-Sum** and **Zero-Sum** games.
- Contrast theoretical Game Theory with computational **Adversarial Search**.
- Construct and evaluate game trees using the **Minimax** algorithm.
- Extend standard 2-player Minimax to handle multiple adversaries.
- Apply **Alpha-Beta Pruning** to drastically reduce search time.
- Formulate **Evaluation Functions** and utilize **Expectimax Search**.

Recap – The Search Landscape

Where we are vs. Where we are going

- **Standard Search (A^*):** Environment is static. We want the shortest path to a known goal.
- **Local Search & CSPs:** Environment is static. We want a valid or optimal final configuration.
- **Game Theory & Adversarial Search:** The environment contains other intelligent, rational agents. They are actively trying to maximize their own utility, which may directly conflict with yours.

The Foundation: What is Game Theory?

The Mathematics of Strategic Interaction

- Game Theory is the study of mathematical models of strategic interaction among rational decision-makers.
- It is the theoretical foundation not just for AI games, but for economics, evolutionary biology, and political science.
- **The Core Premise:** Your optimal decision depends entirely on what you expect the other rational agents to do.

Representing Games: The Payoff Matrix

Simultaneous Decision Making

- In Game Theory, we often model *simultaneous* games (where players move at the same time without knowing the other's choice) using a **Payoff Matrix**.
- The matrix displays the **Utility** (score/payoff) for Player 1 and Player 2 based on their combined choices.

	Player 2: Action A	Player 2: Action B
Player 1: Action A	(P1 Utility, P2 Utility)	(P1 Utility, P2 Utility)
Player 1: Action B	(P1 Utility, P2 Utility)	(P1 Utility, P2 Utility)

Non-Zero-Sum Games: The Prisoner's Dilemma

Where cooperation and betrayal meet

- **Non-Zero-Sum:** The total utility is not constant. Both players can win, or both can lose.
- **Scenario:** Two criminals are interrogated in separate rooms. They can either **Cooperate** (stay silent) or **Defect** (betray their partner).

Prisoners' dilemma		prisoner B	
		confess 	remain silent 
prisoner A	confess 	 5 years  5 years 	 0 year  20 years
	remain silent 	 20 years  0 year 	 1 year  1 year

The Paradox of Rationality

	P2: Confess	P2: Cooperate
P1: Confess	(5 yr, 5 yr)	(0 yrs, 20 yrs)
P1: Cooperate	(20 yrs, 0 yrs)	(1 yrs, 1 yrs)

- Look at the Prisoner's Dilemma matrix. If P2 Cooperates, P1's best move is to Confess (0 yrs vs 1 yr). If P2 Confesses, P1's best move is to Confess (5 yrs vs 20 yrs).
- Confessing is a **Dominant Strategy**. Both rational players will Confess.
- **Nash Equilibrium**: A state where no player can improve their payoff by unilaterally changing their strategy.
- *The Paradox*: The Nash Equilibrium is (5, 5). Even though (1, 1) is better for both, rational agents cannot reach it without a guarantee of trust.

Zero-Sum Games: Pure Competition

I win, you lose.

- **Zero-Sum:** The total utility of the game always adds up to 0 (or a constant). One player's gain is exactly equal to the other player's loss.
- Examples: Chess, Tic-Tac-Toe, Pacman.

	Defense: Block Left	Defense: Block Right
Offense: Attack Left	$(-1, +1)$	$(+1, -1)$
Offense: Attack Right	$(+1, -1)$	$(-1, +1)$

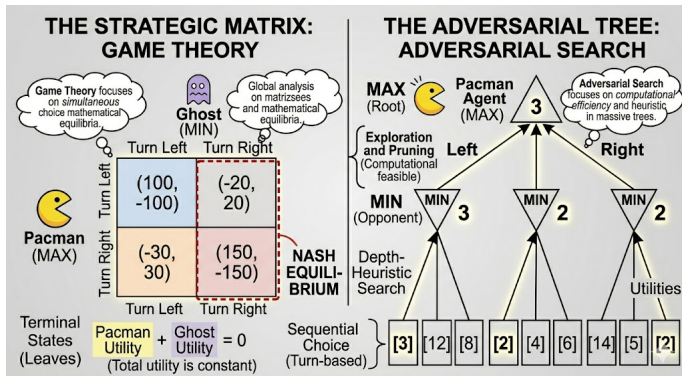
Note: In these games, there is absolutely no room for cooperation. You must assume the opponent wants to destroy your utility.

The Bridge: Game Theory vs. Adversarial Search

From Economics to Computer Science

- **Game Theory** focuses heavily on *simultaneous* games (matrices), non-zero-sum dynamics, and finding mathematical equilibria.
- **Adversarial Search (AI)** focuses on *sequential* games (turn-based, represented as Trees) and the *computational algorithms* required to calculate the next best move when the state space is too massive to solve mathematically.
- Today, we leave the Matrix behind and enter the Game Tree.

Game Theory vs Adversarial Search



Formalizing a Game for AI

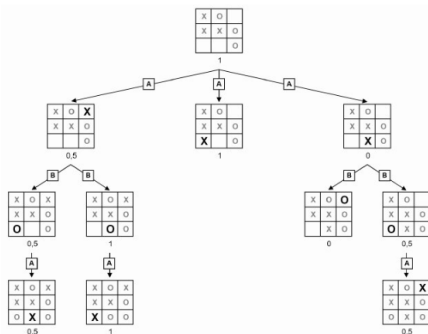
Just like A^* and CSPs, we must formally define the sequential problem space:

- S_0 : The initial state (how the board starts).
- **Player(s)**: Returns which player has the turn in state s .
- **Actions(s)**: Returns the legal moves in state s .
- **Result(s, a)**: The transition model. Returns the new state after action a is taken.
- **Terminal-Test(s)**: Returns true if the game is over (Win/Loss/Draw).
- **Utility(s, p)**: A numeric scoring function for the game outcome at terminal state s for player p .

The Game Tree

Visualizing the Search Space

- We can map out every possible sequence of moves from start to finish.
- **Nodes:** Game states.
- **Edges:** Legal actions.
- The tree alternates between the players' turns until it hits the terminal leaves.

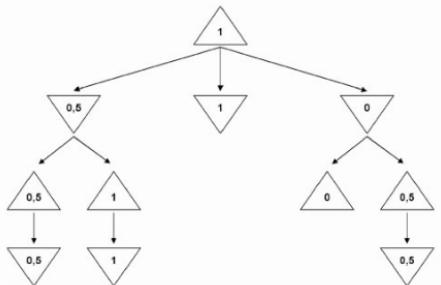


MAX and MIN Nodes

The Two Perspectives

To solve the game tree, we assign a role to each player:

- **MAX (Triangles pointing up Δ):** This is our agent (e.g., Pacman). MAX wants to maximize the final Utility score.
- **MIN (Triangles pointing down ∇):** This is the opponent (e.g., The Ghosts). MIN wants to minimize MAX's Utility score.



The Bottom of the Tree

- The game tree ends at **Terminal States**.
- At these states, the game is over, and the **Utility Function** assigns a hard number.
- Example: Pacman eats all food = +500. Pacman dies = -500.
- *The Problem*: How does MAX know which move to make at the top of the tree, when the scores are only known at the very bottom?

The Minimax Principle

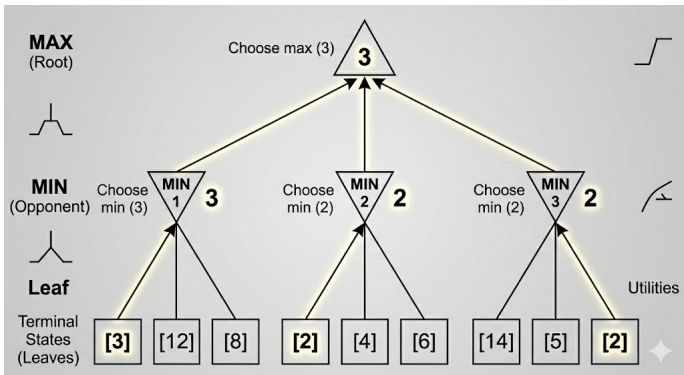
Bottom-Up Reasoning

- We must propagate the terminal utilities back up the tree.
- **The Core Rule:** Assume both players play optimally.
 - When pushing a value up through a MIN node, MIN will always select the *smallest* available value from its children.
 - When pushing a value up through a MAX node, MAX will always select the *largest* available value from its children.

Minimax Execution Trace - Step 1

Expanding the Tree

- First, we expand the entire game tree down to the terminal leaves.
- We calculate the Utility score for every single leaf.



MIN's Turn to Choose

- Look at the level directly above the leaves. These are MIN nodes.
- For the first MIN node (choices: 3, 12, 8), MIN will choose **3**.
- For the second MIN node (choices: 2, 4, 6), MIN will choose **2**.
- For the third MIN node (choices: 14, 5, 2), MIN will choose **2**.

MAX's Turn to Choose

- Now look at the root node. This is a MAX node.
- MAX looks at the values passed up by MIN: **3, 2, and 2**.
- MAX wants the highest score. MAX chooses **3**.
- *Conclusion:* The optimal move for MAX at the root is to go left.

Properties of Minimax

Is it good?

- **Complete?** Yes (if the tree is finite).
- **Optimal?** Yes (against an optimal opponent).
- **Time Complexity:** $O(b^m)$ where b is the branching factor (legal moves) and m is the maximum depth.
- **Space Complexity:** $O(bm)$ because we explore it using Depth-First Search.

Algorithm 1 - Standard Minimax

```
function minimax(node, depth, maximizingPlayer) is
  if depth = 0 or node is a terminal node then
    return the heuristic value of node
  if maximizingPlayer then
    value :=  $-\infty$ 
    for each child of node do
      value := max(value, minimax(child, depth - 1, FALSE))
    return value
  else (* minimizing player *)
    value :=  $+\infty$ 
    for each child of node do
      value := min(value, minimax(child, depth - 1, TRUE))
  return value
```

The Pacman Problem

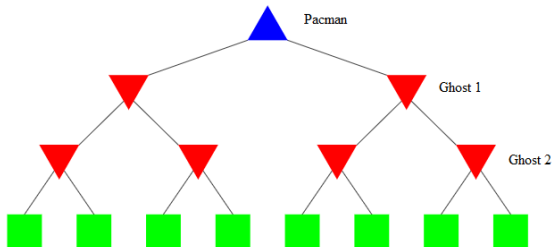
Moving beyond 2 players

- Standard Minimax alternates: $\text{MAX} \rightarrow \text{MIN} \rightarrow \text{MAX} \rightarrow \text{MIN}$.
- But in Pacman, you have 1 Pacman (MAX) and *multiple* Ghosts (MIN 1, MIN 2, MIN 3).
- **A single "Ply" (turn) in Pacman consists of:** Pacman moves, then Ghost 1 moves, then Ghost 2 moves, then Ghost 3 moves.

Multi-Agent Minimax Trees

Stacking the MIN Nodes

- To implement this in your assignment, you cannot just alternate Max-Value and Min-Value.
- You must have a MAX layer, followed by a MIN layer for Ghost 1, followed by a MIN layer for Ghost 2, etc., before you reach the next MAX layer.
- All ghosts act as a collective MIN trying to minimize your score.



Why Minimax Fails in Real Games

- Minimax requires exploring the *entire* game tree to the terminal leaves.
- **Chess:** Branching factor $b \approx 35$, Depth $m \approx 100$.
- The game tree size is 35^{100} .
- To put this in perspective, there are only about 10^{80} atoms in the observable universe. You cannot search the whole tree.

Solution 1: Depth-Limited Search

Cutting the Tree Short

- Since we cannot reach the terminal states, we set a strict depth limit (e.g., $\text{depth} = 3$).
- We expand the tree exactly 3 plies down.
- We stop. But wait, if we aren't at a terminal state, we don't have a Utility score. How do we propagate values up?

Estimating the Future

- We replace the Terminal-Test with a Depth-Limit-Test.
- We replace the Utility function with an **Evaluation Function** (*Eval*).
- An Evaluation Function calculates a heuristic score that estimates the "goodness" of a non-terminal state.
- *Example in Chess*: Queen = 9, Rook = 5, Knight = 3, Pawn = 1. A board with more pieces has a higher evaluation score.

Designing an Evaluation Function

Features and Weights

- A standard evaluation function is a linear combination of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- $f_i(s)$ is a specific feature of the state (e.g., distance to the nearest ghost, amount of food left).
- w_i is the weight (importance) of that feature.
- *Hint for Task 5:* You will need to carefully tune these weights. Distance to food is good (positive weight), but distance to an active ghost is critical (high negative weight).

Solution 2: Alpha-Beta Pruning

Working Smarter, Not Harder

- Even with depth limits, Minimax explores a lot of useless branches.
- **The Idea:** If we find a branch that is demonstrably worse than a choice we have already found elsewhere, we don't need to finish exploring it. We can "prune" it.
- Alpha-Beta Pruning returns the *exact same optimal move* as standard Minimax, but does it in a fraction of the time.

The Intuition of Pruning

A Real-World Example

- You are buying a used car.
- Dealership A offers you a car for \$5,000. This is your baseline (Alpha).
- You go to Dealership B. The salesman says, "Let me calculate the price... it will be at least \$6,000, maybe more depending on fees."
- Do you wait for him to finish the calculation? **No.** You already have a guaranteed \$5,000 option. You "prune" Dealership B and walk away.

Defining Alpha and Beta

The Tracking Variables

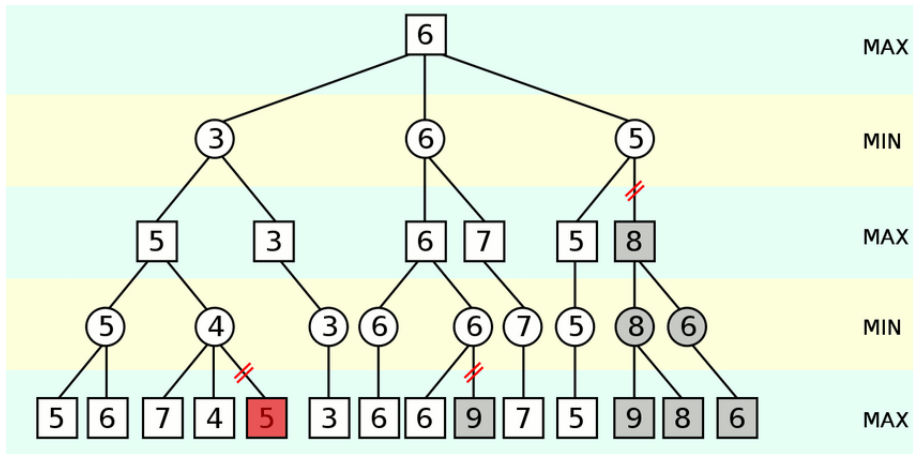
We pass two extra variables down the tree during our DFS:

- α (**Alpha**): The value of the best (highest) choice we have found so far at any choice point along the path for MAX.
- β (**Beta**): The value of the best (lowest) choice we have found so far at any choice point along the path for MIN.

The Pruning Rules

- **MIN Node Rule:** If a MIN node discovers a value v that is less than or equal to α ($v \leq \alpha$), it immediately stops exploring its other children. (MAX will never let the game reach this MIN node anyway).
- **MAX Node Rule:** If a MAX node discovers a value v that is greater than or equal to β ($v \geq \beta$), it immediately stops exploring its other children. (MIN will never let the game reach this MAX node anyway).

Alpha-Beta Trace



Algorithm 2 - Alpha-Beta Pruning

```
function Max-Value(state, alpha, beta)
  if Terminal-Test(state) then return Utility(state)
  v = -infinity
  for each a in Actions(state) do
    v = Max(v, Min-Value(Result(state, a), alpha, beta))
    if v >= beta then return v // Prune!
    alpha = Max(alpha, v)      // Update Alpha
  return v
```

```
function Min-Value(state, alpha, beta)
  if Terminal-Test(state) then return Utility(state)
  v = +infinity
  for each a in Actions(state) do
    v = Min(v, Max-Value(Result(state, a), alpha, beta))
    if v <= alpha then return v // Prune!
    beta = Min(beta, v)        // Update Beta
  return v
```


Important Note on Alpha-Beta for Assignment

Do Not Reorder!

- The efficiency of Alpha-Beta heavily depends on the order you explore the nodes. (If you explore the best moves first, you prune a lot. If you explore worst moves first, you prune nothing).
- **For your assignment:** The autograder expects a very specific number of expanded nodes. You must explore successors in the exact order returned by `getLegalActions`. Do not sort or reorder them, or you will fail the autograder!

What if MIN makes a mistake?

- Both Minimax and Alpha-Beta assume the opponent is a perfectly rational, optimal agent.
- What if you are playing against a human who makes mistakes? Or a Pacman Ghost that chooses its moves using a random number generator?
- If you use Minimax against a random ghost, Pacman will play extremely defensively, hiding in corners, assuming the ghost will always corner him.

Embracing Probability

- We introduce a new type of node: The **Chance Node** (represented by a circle).
- We replace the MIN nodes with Chance nodes.
- Instead of pulling up the absolute *minimum* value, a Chance node calculates the **Expected Utility** (the average of all possible outcomes, weighted by their probability).

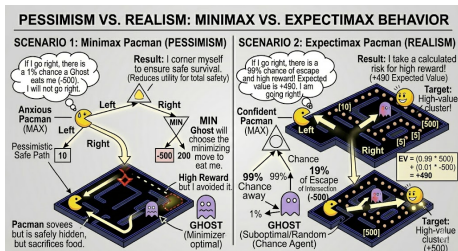
The Math of Chance

- Let's say a Chance node has 3 children with utilities: 10, -5, and 14.
- In Minimax, a MIN node would pull up **-5**.
- In Expectimax, if the ghost chooses randomly, each action has a $\frac{1}{3}$ probability.
- Expected Value = $(\frac{1}{3} \cdot 10) + (\frac{1}{3} \cdot -5) + (\frac{1}{3} \cdot 14) = \frac{19}{3} = \mathbf{6.33}$
- MAX sees a score of 6.33 instead of -5. MAX is now willing to take a calculated risk!

Minimax vs. Expectimax Behavior

Pessimism vs. Realism

- **Minimax:** "If I go left, there is a 1% chance the ghost eats me. The worst-case is death (-500). I will not go left."
- **Expectimax:** "If I go left, there is a 1% chance of death (-500) and a 99% chance of getting 500 points. Expected value is +490. I am going left!"



Algorithm 3 - Expectimax Search

The Pseudocode (For Bonus Task 4)

```
function Exp-Value(state) returns a utility value
  if Terminal-Test(state) then return Utility(state)
  v = 0
  actions = Actions(state)
  p = 1 / length(actions) // Assuming uniform probability

  for each 'a' in actions do
    // Add weighted probability of the MAX child
    v += p * Max-Value(Result(state, a))

  return v
```

(Notice: There is no pruning in standard Expectimax! You cannot prune a branch because every branch mathematically contributes to the final average.)

Wrapping up Adversarial Search

- **Game Theory:** The mathematical foundation (Equilibria, Non-Zero Sum, Matrices).
- **Minimax:** The algorithmic foundation. Safe, optimal, but computationally heavy.
- **Alpha-Beta:** Mathematical pruning that skips branches to save time.
- **Evaluation Functions:** Our heuristic workaround for deep search limits.
- **Expectimax:** A probabilistic approach for dealing with unpredictable adversaries.